



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования

«Иркутский государственный университет»
(ФГБОУ ВО «ИГУ»)

Институт математики и информационных технологий
Кафедра алгебраических и информационных систем

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Научно-исследовательская работа
(получение первичных навыков
научно-исследовательской работы)

Руководитель:
профессор кафедры алгебраических и
информационных систем, д.ф.-м.н.
Винокуров С.Ф.

Студенты:
Ашроев Евгений Вадимович
Шилов Данил Иванович
гр. 02371-ДБ, 3 курс ФИ

Иркутск 2026

Дневник прохождения учебной практики

Дата	Краткое содержание работы	Отметка о выполнении
01.06.26	Получение задания. Задача: поиск минимальной ДНФ недоопределенной булевой функции. Алгоритм: имитация отжига.	
02.06.26 — 03.06.26	Изучение теоретических основ алгоритма имитации отжига и модели термодинамического охлаждения.	
04.06.26	Определение способов представления состояний булевой функции и адаптация операторов изменения конфигураций под АИО.	
05.06.26 — 06.06.26	Программная реализация логики алгоритма на Python и макетирование консольного TUI-интерфейса во фреймворке Textual.	
08.06.26	Интеграция вычислительного модуля с интерфейсной оболочкой, отладка передачи данных через оперативную память.	
09.06.26 — 10.06.26	Тестирование работы алгоритма и подбор оптимального шага охлаждения системы.	
11.06.26	Настройка вывода графиков зависимости целевой функции от итерации и скроллируемого лога векторов в терминале.	
13.06.26	Сбор результатов экспериментов и оформление отчёта по исследовательской практике.	

ВВЕДЕНИЕ

Поиск минимальной дизъюнктивной нормальной формы (ДНФ) недоопределенной булевой функции — это классическая задача дискретной математики и логического проектирования. Она возникает при оптимизации топологии интегральных схем, минимизации логических цепей и автоматизации проектирования цифровых устройств, где требуется сократить количество логических элементов на кристалле. Задача относится к классу NP-трудных. С ростом числа переменных размерность пространства поиска растет экспоненциально, поэтому полный перебор конфигураций быстро становится нереализуемым. Но использование случайных эвристических подходов дает возможность находить качественные решения за вполне приемлемое время.

В данной работе разработано и исследовано программное приложение для оптимизации недоопределенных функций с помощью алгоритма имитации отжига. Этот метаэвристический метод концептуально повторяет термодинамический процесс постепенного охлаждения и кристаллизации металлов, при котором вещество переходит в состояние с минимальной внутренней энергией. Алгоритм моделирует это поведение за счет постепенного снижения расчетной температуры. На первых шагах программа допускает переход к ухудшающим состояниям с вероятностью $P = e^{-D/t_i}$, где D — разность энергий между состояниями. Это помогает вычислениям не застревать в локальных экстремумах, а затем, по мере остывания системы, сфокусироваться на поиске глобального минимума.

Постановка задачи

Рассматривается классическая задача минимизации дизъюнктивной нормальной формы (ДНФ) булевой функции. У нас есть функция $f(x_1, x_2, \dots, x_n)$, заданная наборами своих значений. Нужно найти такую формулу в виде дизъюнкции элементарных конъюнкций, которая будет полностью эквивалентна этой функции, но получит минимальную сложность. А под сложностью здесь понимается суммарное число литералов во всех конъюнкциях выражения.

$$\begin{aligned} &\text{Минимизировать } L(D) = \sum_{j=1}^m |K_j|, \\ &\text{при ограничении } D(\chi) = f(\chi), \quad \forall \chi \in \text{dom}(f), \\ &\text{где } D = K_1 \vee K_2 \vee \dots \vee K_m. \end{aligned}$$

Здесь D — это искомая дизъюнктивная форма, разбитая на m отдельных конъюнкций K_j . Величина $|K_j|$ означает просто количество переменных, входящих в конкретную конъюнкцию, а $L(D)$ выражает итоговую сложность формулы. Наборы переменных χ берутся из области определения функции $\text{dom}(f)$.

Алгоритм имитации отжига

Метод имитации отжига опирается на физическую модель, воспроизводящую поведение кристаллической решетки металла при нагреве и последующем медленном охлаждении [3]. Здесь любое состояние системы S представляет собой конкретное решение задачи, а уровень внутренней энергии $E(S)$ выступает в качестве целевой функции, которую требуется минимизировать. Алгоритм начинает поиск с некоторого случайного варианта и высокой начальной температуры T_0 .

На каждой итерации текущее состояние немного меняется на соседнее. Если энергия нового решения оказывается меньше, система переходит в него сразу. Но если новое состояние увеличивает энергию, переход все равно остается возможным. Вероятность P такого шага рассчитывается по формуле:

$$P = e^{-\frac{\Delta E}{T}},$$

где ΔE — разность энергий между новым и старым состояниями, а T — текущая температура. По ходу работы температура постепенно снижается по линейному закону $T_{k+1} = T_k - \alpha$, где α — шаг охлаждения. Из-за этого вероятность принять ухудшающее решение со временем падает, что помогает алгоритму сначала активно прыгать по разным локальным минимумам, а под конец стабилизироваться в оптимальной точке.

Описание задачи в терминах алгоритма

В нашей задаче поиска минимальной ДНФ мы спроецировали физические понятия на математические. Состояние системы S — это текущая структура дизъюнктивной нормальной формы, которую мы кодируем вектором. Функция энергии g отвечает за длину полученной формулы, то есть считает суммарное количество литералов. Чем формула короче, тем меньше у нее энергия. Функция изменения состояния f берет текущий вектор и инвертирует в нем часть бит, смещая нас в соседнюю точку пространства поиска. А вероятность перехода в ухудшающее состояние вычисляется по классической формуле $P = e^{-D/t_i}$.

Параметры алгоритма

Чтобы программу было удобно тестировать, все стартовые параметры мы вынесли в отдельный конфигурационный файл. Прямо из интерфейса пользователь задает начальную температуру, количество итераций на одном температурном срезе и весовые коэффициенты. В качестве закона изменения температуры мы оставили один рабочий вариант — линейное охлаждение $t_i = t_{i-1} - \alpha$. Сам коэффициент α тоже легко настраивается через элементы TUI.

Реализация алгоритма

Мы разбили проект на два модуля: один человек отвечал за математическую логику, другой — за пользовательский интерфейс. Писали на Python. Для визуала взяли фреймворк Textual [2], он отлично подходит для сборки консольных TUI-приложений.

Мы не стали запускать вычислительное ядро через сторонние процессы вроде `subprocess.run`, потому что это сильно тормозит работу. Вместо этого логика алгоритма напрямую импортируется в память интерфейса. Сами расчеты запускаются асинхронно через `asyncio.to_thread`, поэтому консоль не зависает, пока процессор занят математикой.

Интерфейс подстраивается под окружение пользователя. Библиотека `darkdetect` [1] при запуске опрашивает операционную систему и автоматически включает нужную тему — светлую или темную. Если ОС не дает ответа, программа безопасно откатывается на нашу кастомную темную палитру.

Вычислительные эксперименты

Для апробации мы сгенерировали несколько недоопределенных булевых функций. За ходом поиска можно следить прямо в терминале. Мы встроили

графики, которые рисуются в консоли с помощью псевдографики. На экран в реальном времени выводится кривая зависимости энергии от номера текущей итерации. Ниже расположен скроллируемый лог истории сгенерированных векторов. Чтобы график строился корректно, вычислительный модуль на каждом шаге собирает значения энергии и температуры в отдельные списки и отдает их интерфейсу.

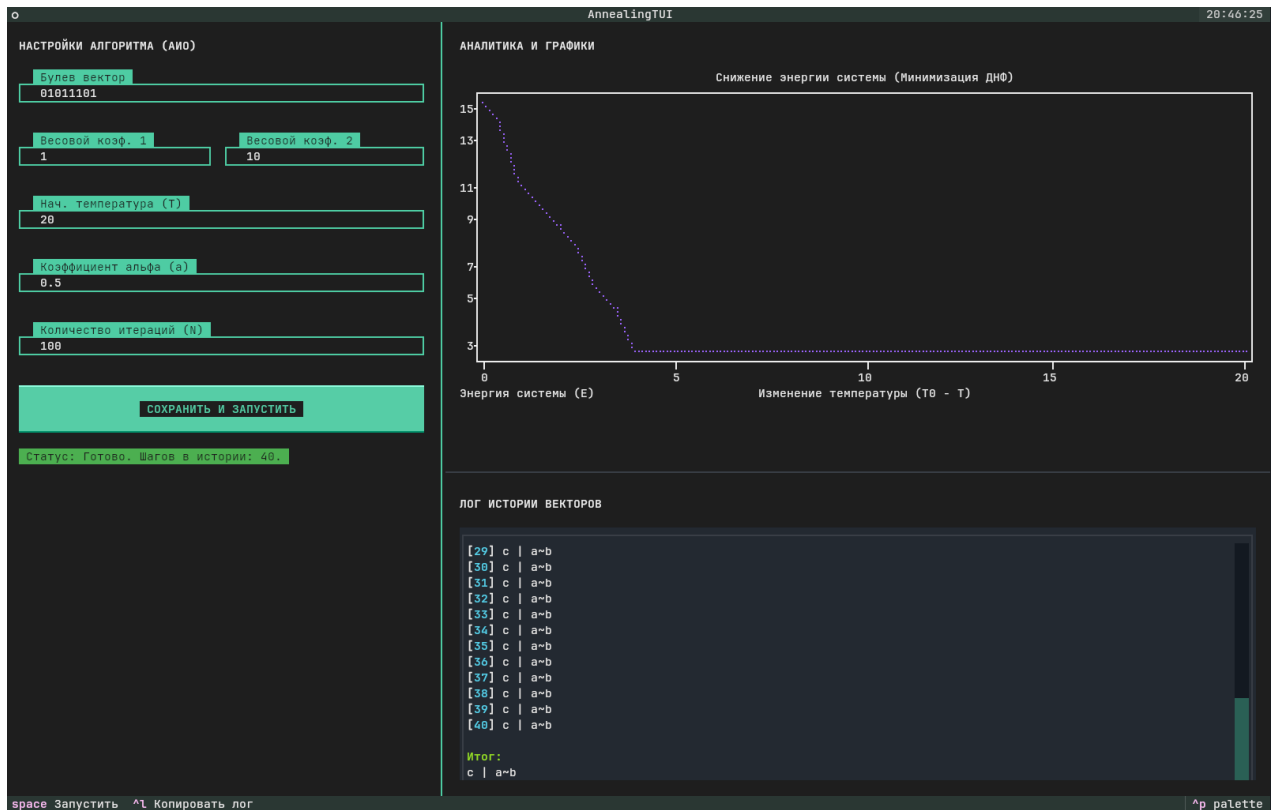


Рисунок 1. Интерфейс программы: график изменения энергии и лог сгенерированных векторов

Мы заметили, что если выставить слишком большой шаг α , система остывает слишком быстро. Алгоритм не успевает перебрать достаточно состояний и застревает в локальных минимумах. А вот при небольшом и плавном шаге охлаждения результаты стабильно улучшаются, особенно на больших значениях начальной температуры.

Тестовые примеры

Для проверки корректности работы алгоритма и тестирования производительности TUI-интерфейса мы использовали набор недоопределенных булевых функций различной размерности. Каждая функция кодировалась в виде бинарного вектора, где длина вектора определяет количество возможных входных наборов (от 4 до 64 значений).

Ниже представлены сгенерированные входные векторы, на которых проводилась отладка программы:

- 1011
- 01011101
- 1001010110011101
- 1011101010111101
- 10111010101111011100111000001001
- 10101010101001100101001001101001
- 1010101010100110010100100110100111100011011010100111011000101101

Итоги работы

Приложение полностью решает поставленную задачу минимизации ДНФ. Командная работа через разделение модулей показала себя отлично: ядро и интерфейс разрабатывались параллельно. Отказ от тяжеловесных способов обмена данными в пользу прямого импорта и асинхронности сделал программу быстрой и отзывчивой. В результате получился удобный консольный инструмент с графиками и автоматической сменой темы оформления.

Список литературы

1. darkdetect. — 2026. — URL: <https://github.com/albertosottile/darkdetect> (дата обр. 13.06.2026).
2. Textual Documentation. — 2026. — URL: <https://textual.textualize.io/> (дата обр. 13.06.2026).
3. Скобцов Ю. А. Основы эволюционных вычислений. — Донецк : ДонНТУ, 2008. — 326 с. — ISBN 978-966-377-056-6.